Faculty of Computer Science and Information Technology

PICKLEBALL AUTOMATED POINT COUNTING USING AI

Wong Tong Yung

87100

Bachelor of Computer Science with Honours

(Network Computing)

2025

ABSTRACT

Pickleball is currently growing at an exponential rate globally. However, a major barrier has emerged due to its unique officiating requirements of pickleball. Pickleball utilizes a complex side-out scoring format where points are only awarded to the serving team, and the score is announced as a unique three-number. Recreational players are subjected to a significant cognitive burden due to this non-linear scoring structure, which frequently results in memory errors, on-court disputes, and a disturbed gameplay flow. While professional tournaments benefit from high-fidelity optical tracking systems like Hawk-Eye, these solutions rely on expensive multi-camera infrastructures that are financially and logistically inaccessible to the average recreational community. Similar to this, current consumer solutions either have ecosystem exclusivity issues or necessitate the purchase of expensive, specialized hardware, creating a need for an affordable substitute. This project involves the creation of an affordable Automated Pickleball Point Counting System that can run fully on a typical consumer laptop. The proposed solution integrates computer vision and artificial intelligence to track match events in real-time using a standard USB webcam. The methodology begins with the custom dataset comprising footage of local gameplay, which is annotated to fine-tune the YOLOv11-m object detection model. The detected object coordinates are subsequently processed by a custom Python-based Game Logic Engine. This engine operates as a finite state machine, enforcing the server rotation and rally termination conditions. Data persistence is managed using local SQLite database, which guarantees offline functionality and facilitates detailed post-match analytics. The final system is accessible through a lightweight web-based dashboard that visualizes the live three-number score and includes a video rewind feature for immediate dispute verification. By validating this prototype against manual scoring methods, the project demonstrates that high-level sports analytics can be democratized. Providing recreational players with a reliable, automated tool that enhances the enjoyment of the sport.

ABSTRAK

Pickleball kini sedang berkembang pada kadar eksponen di peringkat global. Walau bagaimanapun, satu halangan utama telah muncul disebabkan oleh keperluan pengadilan yang unik bagi sukan tersebut. Pickleball menggunakan format pemarkahan side-out yang kompleks di mana mata hanya diberikan kepada pasukan yang melakukan servis, dan skor diumumkan sebagai siri tiga nombor yang unik. Pemain rekreasi sering tertakluk kepada beban kognitif yang ketara disebabkan oleh struktur pemarkahan bukan linear mengakibatkan kesilapan ingatan, pertikaian di gelanggang, dan gangguan kepada kelancaran permainan. Walaupun kejohanan profesional mendapat manfaat daripada sistem penjejakan optik berketepatan tinggi seperti Hawk-Eye, penyelesaian ini bergantung kepada infrastruktur kamera yang mahal, menjadikannya sukar diakses secara kewangan dan logistik oleh komuniti rekreasi biasa. Begitu juga, penyelesaian pengguna semasa sama ada mempunyai isu eksklusiviti ekosistem atau memerlukan pembelian perkakasan khusus yang mahal, sekaligus mewujudkan keperluan untuk alternatif yang mampu milik. Projek ini melibatkan pembangunan sistem pemarkahan pickleball automatik yang kos efektif dan boleh dijalankan sepenuhnya pada komputer riba pengguna biasa. Penyelesaian yang dicadangkan mengintegrasikan visi komputer dan kecerdasan buatan untuk menjejak peristiwa perlawanan dalam masa nyata menggunakan kamera web USB standard. Sistem akhir boleh diakses melalui papan pemuka berasaskan web yang ringan, yang memvisualisasikan secara langsung dan menyertakan ciri putar semula video untuk pengesahan segera sekiranya berlaku pertikaian. Dengan mengesahkan ketepatan prototaip ini berbanding kaedah pemarkahan manual, projek ini membuktikan bahawa analitik sukan peringkat tinggi boleh didemokrasikan, sekaligus menyediakan pemain rekreasi dengan alat automatik yang boleh dipercayai yang meningkatkan keseronokan sukan tersebut.

TABLE OF CONTENTS

ABSTRACT ii

ABSTRAK iii

TABLE OF CONTENTS iv

LIST OF FIGURES vii

LIST OF TABLES ix

CHAPTER 1 INTRODUCTION 1

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Scope 3

1.4 Objective 4

1.5 Brief Methodology 4

1.6 Significance of Project 6

1.7 Project Schedule 7

1.8 Expected Outcome 7

CHAPTER 2 : LITERATURE REVIEW 8

2.1 Introduction 8

2.2 Pickleball Scoring 8

2.3 Review of Existing Commercial Systems 10

2.3.1 Hawk-Eye 11

2.3.2 Swingvision 13

2.3.3 In/Out 15

2.4 Comparison of Existing Commercial Solutions 16

2.5 Object Detection Algorithms 18

2.5.1 Two-Stage Detectors (Faster R-CNN) 19

2.5.2 One-Stage Detectors (SSD) 20

2.5.3 You Only Look Once (Yolov11) 21

2.5.4 Comparison of Algorithms 22

2.6 Comparative Summary 23

CHAPTER 3 : REQUIREMENT ANALYSIS AND DESIGN 24

3.1 Introduction 24

3.2 Methodology 25

3.2.1 Phase 1: Data Collection and Preparation 27

3.2.2 Phase 2: Core Detection Module (Yolov11 Training) 29

3.2.3 Phase 3: Game Logic And Database Development 29

3.2.4 Phase 4: System Integration And Testing 29

3.3 Requirements Analysis 30

3.3.1 Functional Requirements 32

3.3.2 Non-Functional Requirements 34

3.3.3 Hardware and Software Requirements 35

3.4 System Design 37

3.4.1 Use Case Diagram 38

3.4.2 Use Case Description 39

3.4.3 Sequence Diagram 45

3.4.4 Activity Diagram 46

3.4.5 Class Diagram 48

3.4.6 Entity Relationship Diagram (ERD) 49

3.4.7 Graphical User Interface (GUI) Design 50

3.5 Summary 51

REFERENCES 52

LIST OF FIGURES

Figure 1.1 Waterfall model diagram.....5

Figure 1.2 Overview of Gantt Chart.....7

Figure 2.1 Illustration of the three-number scoring format.....9

Figure 2.2 The Side-Out Scoring Process 10

Figure 2.3 Operational diagram of the Hawk-Eye tracking system (Hawk-Eye Innovations, n.d.) 11

Figure 2.4 A Hawk-Eye camera mounted on a stadium gantry (Daily Mail, 2013).....12

Figure 2.5 Swing stick phone mount setup. (SwingVision, 2025).....13

Figure 2.6 Price of the subscription fee.....14

Figure 2.7 Setup of the In/Out Net Device V4.0 (In/Out Tennis, n.d.) 15

Figure 2.8 High-level architecture of Faster R-CNN (GHOURY et al., 2019).....19

Figure 2.9 SSD architecture showing detection at multiple feature map scales (Li et al, 2021) ...20

Figure 2.10 The YOLO network architecture (Redmon et al., 2016) 21

Figure 3.1 Proposed System Architecture 24

Figure 3.2 Development Framework of the Automated Pickleball Point Counting System.....26

Figure 3.3 The position of camera at the rear baseline 27

Figure 3.4 Illustration of the bounding box annotation workflow using a proxy object for preliminary testing.....28

Figure 3.5 System Modeling Language Diagram 31

Figure 3.6 Automated Pickleball Point Counting System use case diagram 38

Figure 3.7 Sequence diagram of Automated Pickleball Point Counting System 45

Figure 3.8 Activity Diagram of Pickleball Scoring Rules.....47

Figure 3.9 Class Diagram of the Automated Pickleball Point Counting System 48

Figure 3.10 Entity Relationship Diagram of the System Database 49

Figure 3.11 Graphical User Interface of scoreboard 50

LIST OF TABLES

Table 2.1 Comparison of existing commercial solutions 17

Table 2.2 Comparison of Object Detection Algorithms 22

Table 3.1 Table 3.1 List of Functional Requirements 32

Table 3.2 List of Non-Functional Requirements 34

Table 3.3 Hardware Requirements 35

Table 3.4 Software Requirements 36

Table 3.5 Use Case Description for automated game tracking 39

Table 3.6 Use Case Description for Process Object Detection 40

Table 3.7 Use Case Description for Update Game State 41

Table 3.8 Use Case Description for Save Match Log 42

Table 3.9 Use Case Description for View Live Dashboard 43

Table 3.10 Use Case Description for Rewind Video 44

CHAPTER 1 INTRODUCTION

1.1 Introduction

Pickleball is a rapidly growing paddle sport that attracts a lot of newcomers due to its easy learning curve. The origin of the pickleball was invented in 1965 in Washington, USA (Brinkerhoff, 2023). The game is a combination of few elements such as tennis, badminton, and ping pong. The court is played on a badminton-sized court with a slightly lower net similar to tennis while using an oversized paddle to hit a polymer ball. The game usually plays in doubles since it is the most common format in both recreational and competitive play.

Pickleball game consists of few serving rules to decide the scoring. The serves must starts with underhand with the paddle below the waist. Ball must travel diagonally passing the non-volley zone and bounce once by both sides. This rule is named as two-bounce rule. If a player volleys within the kitchen, hits the ball into the net, or out of bounds. It is considered as fault that leads to the end of the rally.

The common scoring system in pickleball is known as traditional side-out scoring. This is the primary scoring system adopted by the official tournament and used as a default in recreational play. The scoring mechanics awards points to the serving side if wins a rally. But if the receiving team wins, they will win a serve instead of the points (Mackie, 2024). A lot of confusion came from the doubles scoring which requires tracking three numbers. Which is the serving teams score, the receiving teams score, and the server number. Before serving, the server must announce these three-number scores. This increases the difficulty for players to remember the numbers, especially when there are long, intense rallies or distractions on or off the court.

The reliance on manual scorekeeping often leads to disrupting gameplay, increasing the likelihood of prone human error. This often led to frequent disputes that disrupt the flow and enjoyment of the game. Highlighting a clear need for a technological solution to automate the process.

1.2 Problem Statement

Pickleball is the fastest-growing sport in Malaysia, primarily due to the low barrier of entry and highly social sport. This has led to significant growth in new recreational and amateur players in recent years (Elara, 2023).

However, the supporting technology for the game has not kept pace with the current rapid growth of players. For the vast majority of recreational and amateur matches, scorekeeping remains a manual process. This system is highly prone to human error, indicating players are overload with the information since players need to track the three-number score, server rotation, and game state simultaneously while actively playing the pickleball.

The unreliability of this manual process frequently leads to disputes over the score. These conflicts not only disrupt the flow of play but also diminish the overall enjoyment of the players. While professional-grade automated systems like Hawk-Eye exist in the market, these systems are not available for vast majority of the pickleball community. They are purposely designed for professional broadcast tournaments, not for recreational and community courts.

Therefore, this project addresses the issue of lack of accessible technology between the recreational and amateur pickleball community. The aim is to develop a low-cost, reliable, and

user-friendly automated point counting system for average players enhancing the players overall enjoyment and experience.

1.3 Scope

The scope of this project is to develop an Automated Pickleball Point Counting System for recreational pickleball running on a local PC. The object detection module using YOLOv11-m model and fine-tuning using the PyTorch framework. This model will be implemented in Python, using the OpenCV library to process the video feed. This project will be using SQLite database to store the game events with a timestamp. The simple graphical user interface to display in dashboard will be built with standard HTML, CSS, and JavaScript, running in a browser.

Due to the difficulty of tracking the player pose movement, this project will be limited to tracking the ball only. The foot faults or kitchen violations will not be included in the system. Currently, a single camera will be used to setup in the between on the court to detect the movement of the ball.

1.4 Objective

- To design an Automated Pickleball Point Counting System by identifying real-time object detection models and defining the complete system architecture.
- To develop a functional prototype of the system by training and fine-tuning a YOLOv11-m model on a custom dataset and integrating with a Python-based game logic engine.
- To evaluate the accuracy and reliability of the Automated Pickleball Point Counting System prototype by comparing the results against manual scorekeeping.

1.5 Brief Methodology

This project adopts the Waterfall model as the primary Software Development Life Cycle (SDLC). This model follows the sequential approach for the development of stages, ensuring that requirements data are developed and recorded before proceeding to implementation and testing. The project execution is organized into the following four phases which are data collection and preparation, computer vision model implementation, game engine and database construction and system integration and testing. Figure 1.1 below show the Waterfall diagram in the sequential approach.

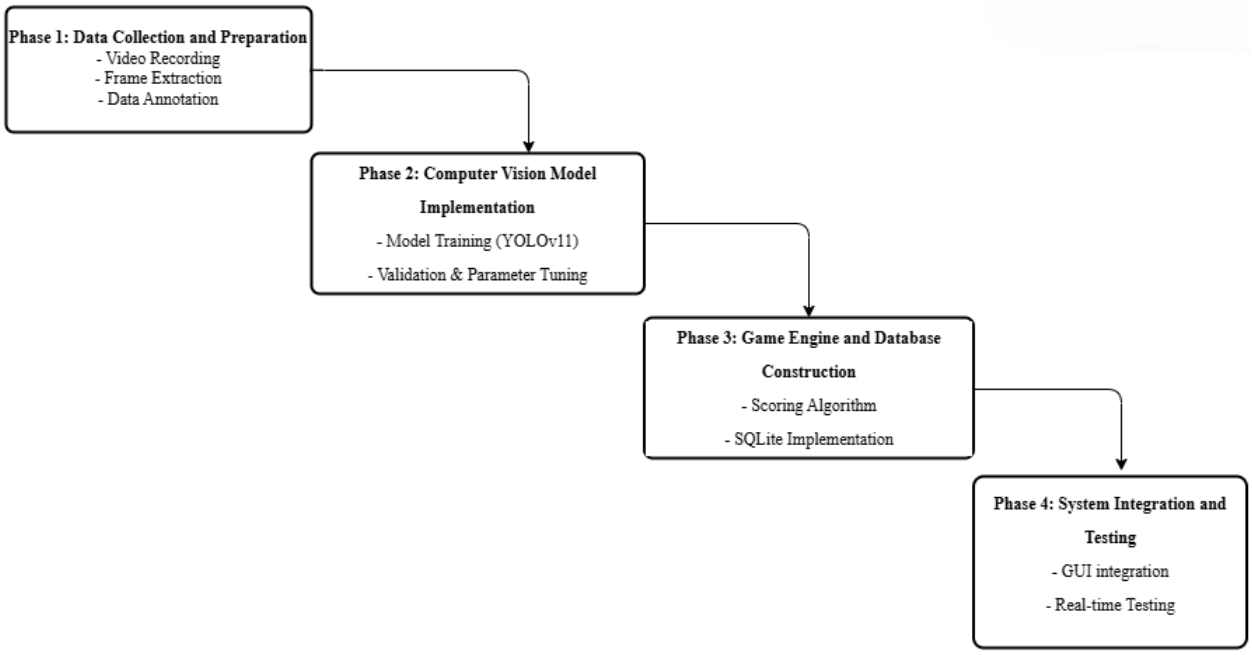

Figure 1.1 Waterfall model diagram

The development process begins with data collection and preparation. A custom dataset will be created by recording local pickleball games. Raw video footage will be processed into individual frames, which will be annotated with bounding boxes to create training data.

This next phase followed by the computer vision model implementation. YOLO object detection model will be fine-tuned on the custom dataset using the PyTorch framework. The training process in this stage will focus on maximizing mean Average Precision to ensure the model reliably outputs accurate real-time ball coordinates.

The third phase involves game engine and database construction. A Python-based game logic engine will be designed to manage side-out scoring rules, track server rotation, and maintain the overall game state. Additionally, an SQLite database will be implemented to log all game events with timestamps.

Finally, the process concludes with system integration and testing. The coordinates are fed to the logic engine to update the score and log events, allowing for real-time dashboard display and post-match analytics. This phase will be performed in real-time testing. System will be validated in a live match environment to compare the scoring accuracy against manual scorekeeping.

1.6 Significance of Project

The primary significance of this project is to enhance the player experience. By automating the process of manual scoring, the system effectively reduces human error, which minimizes on-court disputes and game interruptions. This allows players to remain fully focused on the game, thereby improving the overall experience and enjoyment of the sport.

Furthermore, the project plays a crucial role in democratizing sports technology. It addresses a significant accessibility gap by demonstrating that a low-cost, software-based solution provides a viable alternative to expensive professional systems. This approach makes advanced sports analytics accessible to the wider recreational community.

1.7 Project Schedule

	Name	Duration	Start	Finish
1	**1. FYP1**	**61 days?**	**10/20/25 8:00 AM**	**1/12/26 5:00 PM**
2	**1.2 Brief Proposal**	**6 days**	**10/20/25 8:00 AM**	**10/27/25 5:00 PM**
3	1.2.1 Finalize Project Title	1 day	10/20/25 8:00 AM	10/20/25 5:00 PM
4	1.2.2 Confirmation with Supervisor	1 day	10/20/25 8:00 AM	10/20/25 5:00 PM
5	1.2.3 Identify Basic Skill Requirements	1 day	10/21/25 8:00 AM	10/21/25 5:00 PM
6	1.2.4 Draft Description of the Project	1 day	10/21/25 8:00 AM	10/21/25 5:00 PM
7	1.2.5 Define Problem Statement	2 days	10/22/25 8:00 AM	10/23/25 5:00 PM
8	1.2.6 Establish Objectives	2 days	10/23/25 8:00 AM	10/24/25 5:00 PM
9	1.2.7 Outline Project Outcome	1 day	10/24/25 8:00 AM	10/24/25 5:00 PM
10	1.2.8 Gather Key References	1 day	10/24/25 8:00 AM	10/24/25 5:00 PM
11	1.2.9 Submission of Brief Proposal	0 days	10/25/25 8:00 AM	10/27/25 5:00 PM
12	**1.3 Develop Full Proposal**	**12 days**	**10/27/25 8:00 AM**	**11/11/25 5:00 PM**
13	1.3.1 Refine Project Title	1 day	10/27/25 8:00 AM	10/27/25 5:00 PM
14	1.3.2 Write Introduction	1 day	10/28/25 8:00 AM	10/28/25 5:00 PM
15	1.3.3 Write Problem Statement	2 days	10/28/25 8:00 AM	10/29/25 5:00 PM
16	1.3.4 Define Project Scope	2 days	10/30/25 8:00 AM	10/31/25 5:00 PM
17	1.3.5 Detail Aims and Objectives	2 days	10/30/25 8:00 AM	10/31/25 5:00 PM
18	1.3.6 Outline Brief Methodology	2 days	11/1/25 8:00 AM	11/4/25 5:00 PM
19	1.3.7 Explain Significance of Project	2 days	11/4/25 8:00 AM	11/5/25 5:00 PM
20	1.3.8 Create Project Schedule	1 day	11/7/25 8:00 AM	11/7/25 5:00 PM
21	1.3.9 Describe Expected Outcome	1 day	11/8/25 8:00 AM	11/10/25 5:00 PM
22	1.4.0 Milestone: Submission of Full Proposal	1 day	11/11/25 8:00 AM	11/11/25 5:00 PM
23	**1.4 Write Chapter 1**	**6 days?**	**11/10/25 8:00 AM**	**11/17/25 5:00 PM**
24	1.4.1 Finalize Introduction	4 days	11/10/25 8:00 AM	11/13/25 5:00 PM
25	1.4.2 Submission of Chapter1	1 day	11/15/25 8:00 AM	11/17/25 5:00 PM
26	1.4.3 Milestone: Chapter 1 Draft Submitted	1 day?	11/15/25 8:00 AM	11/17/25 5:00 PM
27	**1.5 Write Chapter 2**	**16 days?**	**11/17/25 8:00 AM**	**12/8/25 5:00 PM**
28	1.5.1 Reasearch Pickleball Rules & Scoring	4 days	11/17/25 8:00 AM	11/20/25 5:00 PM
29	1.5.2 Analyze Existing Commercial Solutions	6 days?	11/17/25 8:00 AM	11/24/25 5:00 PM
30	1.5.3 Compare Object Detection Algorithms	10 days?	11/24/25 8:00 AM	12/5/25 5:00 PM
31	1.5.4 Draft Comparative Summary Tables	1 day?	12/7/25 8:00 AM	12/8/25 5:00 PM
32	1.5.5 Milestone: Submission of Chapter2	1 day	12/7/25 8:00 AM	12/8/25 5:00 PM
33	**1.6 Write Chapter 3**	**16 days?**	**12/8/25 8:00 AM**	**12/29/25 5:00 PM**
34	1.6.1 Define Methodology Phases	5 days	12/8/25 8:00 AM	12/12/25 5:00 PM
35	1.6.2 List Functional & Non-Functional Requirements	1 day?	12/13/25 8:00 AM	12/15/25 5:00 PM
36	1.6.3 Prepare The System Design	10 days?	12/15/25 8:00 AM	12/26/25 5:00 PM
37	1.6.4 Submission of Chapter3	1 day	12/28/25 8:00 AM	12/29/25 5:00 PM
38	1.6.5 Milestone: Chapter 3 Draft Submitted	1 day?	12/28/25 8:00 AM	12/29/25 5:00 PM
39	**1.7 Prepare for FYP Symposium**	**1 day**	**1/9/26 8:00 AM**	**1/9/26 5:00 PM**
40	1.7.1 Create and Deliver Online Presentation	1 day	1/9/26 8:00 AM	1/9/26 5:00 PM
41	**1.8 Complete Finalisation**	**1 day?**	**1/12/26 8:00 AM**	**1/12/26 5:00 PM**
42	1.8.1 Submission of Final Report	0 days	1/12/26 5:00 PM	1/12/26 5:00 PM
43	1.8.2 Milestone: Final Report is submitted	1 day?	1/12/26 8:00 AM	1/12/26 5:00 PM

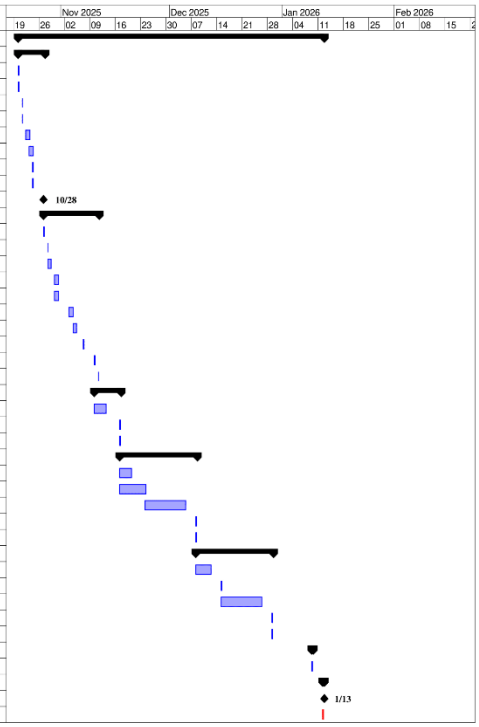

Figure 1.2 Overview of Gantt Chart

1.8 Expected Outcome

The primary delivery of this project will be a functional system prototype, which is a complete, integrated hardware and software solution capable of automatically counting points for a pickleball match. The core components consist of custom-annotated pickleball video dataset, fine-tuned YOLO model, Python-based game logic engine and the user dashboard.

CHAPTER 2 : LITERATURE REVIEW

2.1 Introduction

This chapter establishes the theoretical and technical foundations necessary for developing an Automated Pickleball Point Counting System. The review begins by analyzing the rules of pickleball, specifically emphasizing the complexity of the traditional side-out scoring format. Subsequently, it conducts existing commercial solutions, from professional multi-camera systems like Hawk-Eye to mobile applications like SwingVision. This comparison highlights the accessibility and cost limitations for recreational players. Finally, the chapter reviews relevant academic research in computer vision to justify the choice of the YOLO object detection model for high-speed ball tracking.

2.2 Pickleball Scoring

In pickleball tournaments, the most widely adopted format is the three-number scoring System. Unlike tennis or badminton, which use rally scoring, pickleball awards a point for each rally won. Doubles pickleball adopts a unique side-out scoring format compared to others sport. In this system, only the serving team can score points when they win a rally. If the receiving team wins a rally, they do not score a point but gain the right to serve.

This format introduces a unique complexity where the score is announced as three distinct numbers. These numbers represent the serving teams score, the receiving teams score, and the server number, indicating which partner is serving. Figure 2.1 illustrate the components of the

three-number scoring format. The first number represents the serving teams score, second is receiving teams score, and the third indicates which server is currently serving.

Figure 2.1 Illustration of the three-number scoring format

This system can be challenging for players, especially recreational players who want to focus on the game. The difficulty arises because players must track these three variable numbers simultaneously while actively playing. For instance, the server number applies only to the current service rotation, and players frequently forget whether they are the first or second server after a long rally. Furthermore, because the receiving team earns the serve rather than points, the scoring progression is non-linear, often leading to on-court disputes and forcing players to guess the forgotten scores. As a result, this reliance on manual memory disrupts the flow of the game. This highlights a clear need for an automated solution that can track the game state independently.

To illustrate the rotation process, Figure 2.2 shows the complete decision flow for side-out scoring. The diagram begins with the match score at 0-0-2, indicating the serving team switch partner and having another chance to serve. if the serving team wins a rally, they add a point and

switch sides. Conversely, if the receiving team wins, no point is score and server will be decided whether moves to the partner or results in a side out to the opponents.

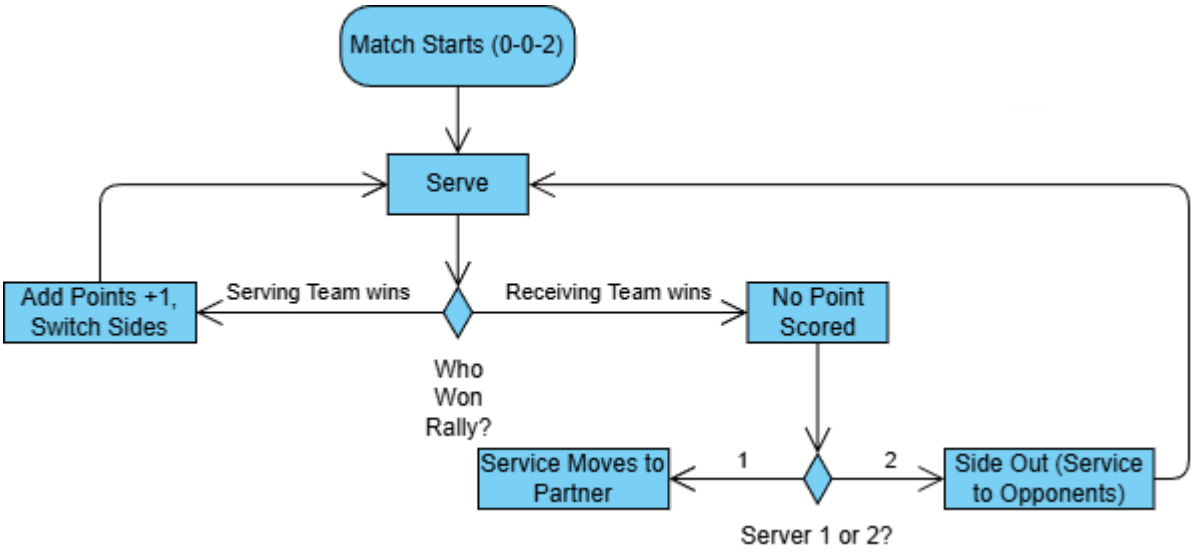

Figure 2.2 The Side-Out Scoring Process

2.3 Review of Existing Commercial Systems

The integration of Artificial Intelligence (AI) and Computer Vision has dramatically changed the landscape of sports officiating and analytics. Current market solutions for automated tracking are generally divided into three tiers based on cost, infrastructure complexity, and the intended user. These tiers include professional optical tracking systems used in tournaments, consumer mobile applications, and dedicated portable hardware designed for recreational use. The following sections will examine specific examples within each category to highlight their limitations regarding the recreational pickleball community.

2.3.1 Hawk-Eye

Advanced optical tracking technologies are currently the benchmark for sports officiating, particularly in high-profile competitions like Grand Slams and the Olympics. The Hawk-Eye system is the preferred choice for these professional events because it produces highly accurate ball tracking using a multi-camera array. It works by synchronizing video feeds from 6 to 10 high-performance cameras placed around the stadium, as shown in Figure 2.3. These cameras generate a three-dimensional model of the court to triangulate the precise position of the ball, allowing the creation of virtual replays used in challenge systems (Hawk-Eye Innovations, n.d.).

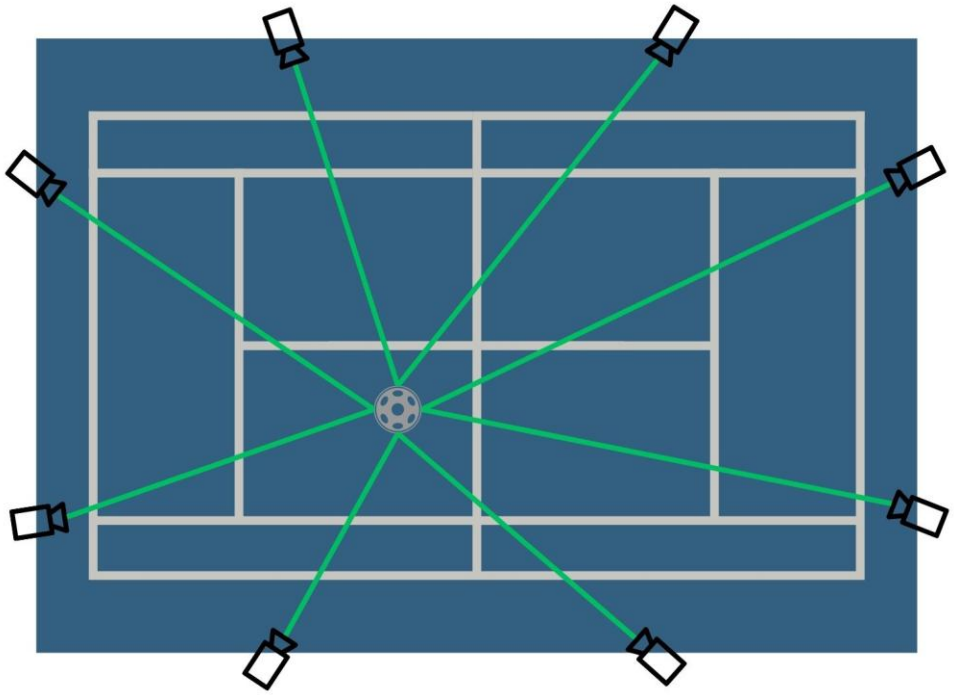

Figure 2.3 Operational diagram of the Hawk-Eye tracking system (Hawk-Eye Innovations, n.d.)

Despite the advantages of the Hawk-Eye system, the Hawk-Eye system remains largely inaccessible for recreational application due to its dependency on heavy infrastructure. As

illustrated in Figure 2.4, a standard installation required permanent cabling, elevated camera mounting, and on-site data processing facilities. Moreover, the financial barrier is high, Faiza (2025) estimates installation costs at approximately $100,000 per venue, exclusive of ongoing licensing and maintenance fees. Therefore, while such systems are effective for professional broadcasting, they are not financially and logistically viable for implementation by community centers or individual stakeholders.

Figure 2.4 A Hawk-Eye camera mounted on a stadium gantry (Daily Mail, 2013)

2.3.2 Swingvision

In the consumer market, SwingVision stands out as the leading solution. This is a mobile application that uses the advanced computing power (NPU) of modern smartphones to track shots and referee matches. Unlike professional systems that rely on complex array of multiple cameras, SwingVision works using footage from a single camera, making it more accessible to casual players.

The parameters are simple, as shown in Figure 2.5. Users simply mount their smartphones high on the rear fence using a tripod or special swing stick. From this perspective, the artificial intelligence of SwingVision analyzes the video stream in real time, detecting the lines of the field and tracking the trajectory of the ball.

Figure 2.5 Swing stick phone mount setup. (SwingVision, 2025)

Although SwingVision provides a great solution to tackle the issue, there are significant obstacles that limit its use by casual players. The issue is the app relies entirely on Apple technology, which creates an ecosystem lock. This requirement excludes a large portion of the users who own Android devices. Moreover, SwingVision runs on a subscription model. Critical features like automated scoring are locked behind a paywall, costing users to pay approximately $14.99/month or $179.99/year. Even with the introductory offers shown in the figure 2., the price proposes a burden to casual players.

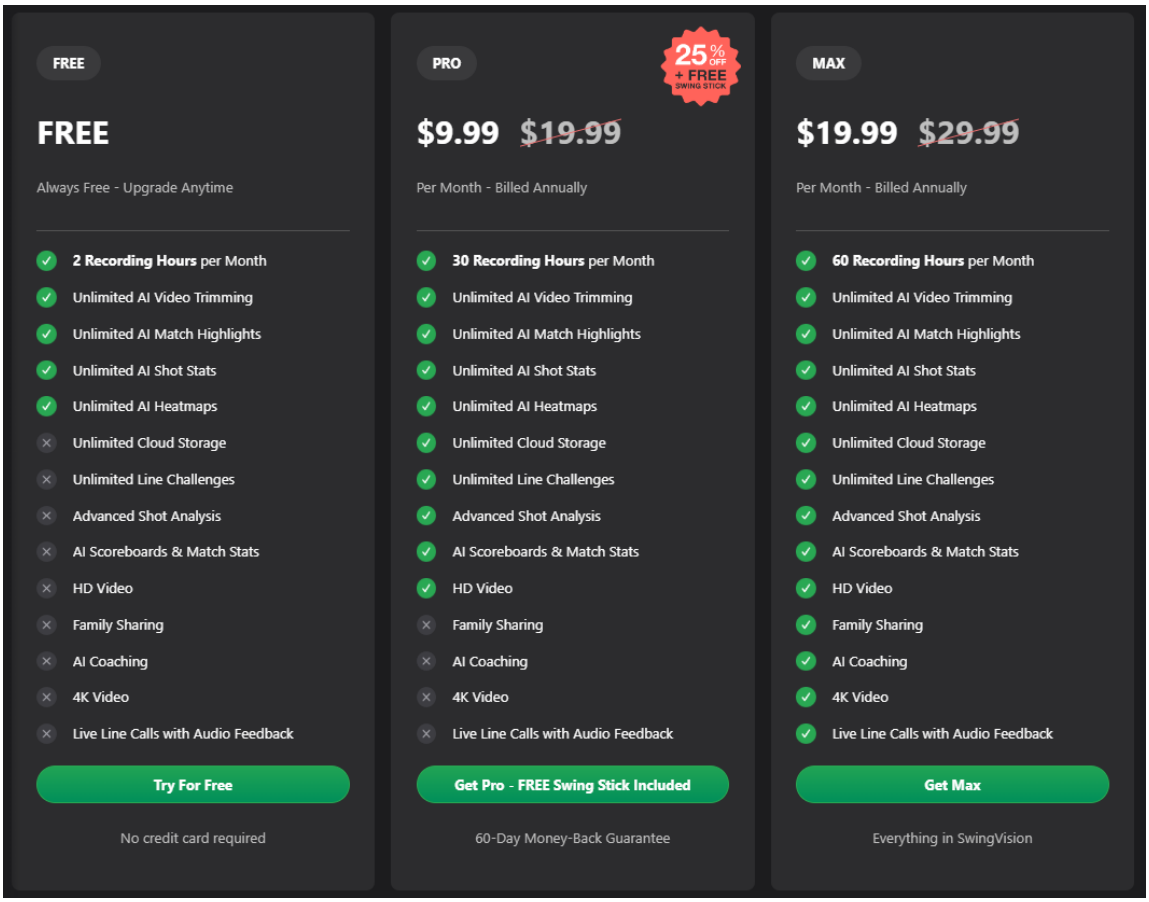

Figure 2.6 Price of the subscription fee

2.3.3 In/Out

The In/Out device is a dedicated portable hardware unit designed specifically for automated line calling. Roughly the size of a GoPro, it functions as a standalone dual-camera system that mounts directly onto the net post or using the device stand to monitor the court. As illustrated in Figure 2.7, the device utilizes two wide-angle cameras to monitor the court lines. When a ball lands, the device processes the video locally and emits a beep sound and a light flash if the ball is determined to be out. The recommended setup by manufacturer is to buy dual device so can minimize blind spots caused by players blocking the camera view.

Figure 2.7 Setup of the In/Out Net Device V4.0 (In/Out Tennis, n.d.)

Despite its automation, the In/Out system presents practical barriers for recreational use. First is the cost-to-value ratio, a recommended dual-unit setup for pickleball exceeds $500 USD. Unlike a laptop or smartphone, this is a significant cost for single-purpose hardware. Second is the

operational friction, users must charge and calibrate the device before every match, with performance often degrading in variable lighting conditions. Finally, attaching a fence to a post creates a physical hazard. Given the aggressive nature of pickleball games near the kitchen, placing the equipment inside the net runs the risk of being hit by rackets and being a distraction, unlike the proposed system that ensures equipment safety off the court.

2.4 Comparison of Existing Commercial Solutions

To clearly define the market landscape, a direct comparison was made against the commercial solutions reviewed in this chapter. Table 2.1 evaluates each system based on cost, hardware complexity, and accessibility for the average player.

Table 2.1 Comparison of existing commercial solutions

Feature	**Hawk-Eye (Hawk-Eye Innovations, n.d.)**	**SwingVision (SwingVision, 2025)**	**In/Out (In/Out Tennis, n.d.)**
Primary Target	Official Tournaments	Amateur / Recreational Players	Club / Line Calling
Hardware Required	Multi-Camera Array (6+)	High-End iPhone (A12+)	Dual Dedicated Cameras
Setup Complexity	High, Permanent installation required	Low, Mount phone on fence	Medium, Mount device on net post
Cost Model	Extreme (> RM260,000)	Subscription (~RM800/year)	Hardware Buy (~RM2200)
Platform	Proprietary Server	IOS Ecosystem Only	Embedded System
Main Limitation	Infrastructure Dependent	Subscription & Hardware Lock	Calibration required

As shown in Table 2.1, there is a clear trade-off between accuracy and affordability in today's market. Professional solutions such as the Hawk-Eye Innovations (n.d.) system provide the high precision required for officiating but demand a financial and logistical infrastructure that is unreachable for community use. On the consumer side, apps like SwingVision (2025) reduce costs but remain limited by ecosystem lock-in and subscription models. Similarly, dedicated hardware solutions such as In/Out Tennis (n.d.) solve the subscription problem, but introduce another issue of requiring users to purchase, charge, and calibrate specialized equipment for every match.

Therefore, a significant gap remains for systems that can provide reliable automated tracking on standard hardware without the monthly subscriptions or proprietary devices. This project addresses that challenge by implementing the YOLOv11 object detection algorithm to

provide a real-time, low-cost alternative for recreational pickleball players using only a generic laptop.

2.5 Object Detection Algorithms

Object detection is a fundamental computer vision task that involves identifying and locating instances of semantic objects of a certain class in digital images or videos (Jiao et al., 2019). While image classification focuses on categorizing the images into a label, object detection extends this capability by simultaneously identifying the position and labelling the object (Liu et al., 2019). It transforms from identifying instance of the object to precisely determining the coordinate and classes of the object simultaneously.

The core complexity of object detection lies in the execution of two different tasks in classification and localization. Classification is responsible for determining the identity of the object and labeling it as a class. On the other hand, localization requires the model to precisely estimate the geometric coordinates of a bounding box of an object. Both requirements combined required significant computational power to process compared to the standard image classification. The model cannot simply assign a label to the whole image instead needs to scan the entire frame to identify the geometric coordinates.

In the domain of deep learning, object detection algorithms are generally categorized into two primary architectures which are two-stage detectors and one-stage detectors (Zhao et al., 2019). One-stage detectors prioritize inference speed by performing both tasks simultaneously. Two-stage detectors prioritize in accuracy by separating the localization and classification steps. To determine the most suitable architecture for real-time pickleball tracking, few algorithms are reviewed and suggested as a most potential algorithm such as Faster R-CNN, SSD, and YOLO.

2.5.1 Two-Stage Detectors (Faster R-CNN)

The Region-based Convolutional Neural Network (R-CNN) family constitutes a foundational approach in deep learning-based object detection. Its most prominent iteration, Faster R-CNN, functions through a distinct two-phase process , as illustrated in Figure 2.8. In the first phase, Region Proposal Network (RPN) scans the image to generate the object bounding boxes. In the second phase, classifies the object and refines the bounding box coordinates to ensure precision (Jiao et al., 2019).

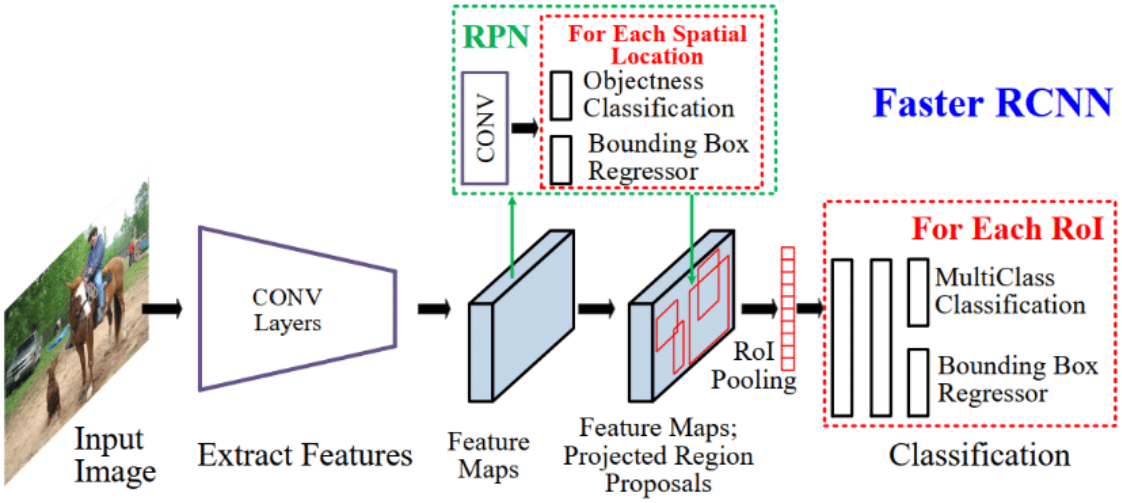

Figure 2.8 High-level architecture of Faster R-CNN (GHOURY et al., 2019)

Despite Faster R-CNN being able to offer a high accuracy performance, this two-stage architecture presents a limitation for the pickleball tracking project. Computational is very expensive when processing a large amount of data. Typically yielding speeds of only 5–7 Frames Per Second (FPS) on standard consumer laptops (Ren et al., 2017). This latency is unacceptable for the project need to track a real-time application such as high velocity pickleball, which can travel at speeds exceeding 60 km/h (Fischetti, 2024). In a low frame rate would result in significant motion gaps between detections.

2.5.2 One-Stage Detectors (SSD)

The Single Shot MultiBox Detector (SSD) was developed to address the issue of latency bottlenecks found in two-stage architectures like Faster R-CNN. It eliminates the use of the region proposal stage entirely and treats object detection as a single deep neural network regression problem (Jiao et al., 2019). Moreover, SSD can predict bounding boxes and class probabilities directly from feature maps in a single forward pass of the network, significantly reducing computational overhead (Zhao et al., 2019).

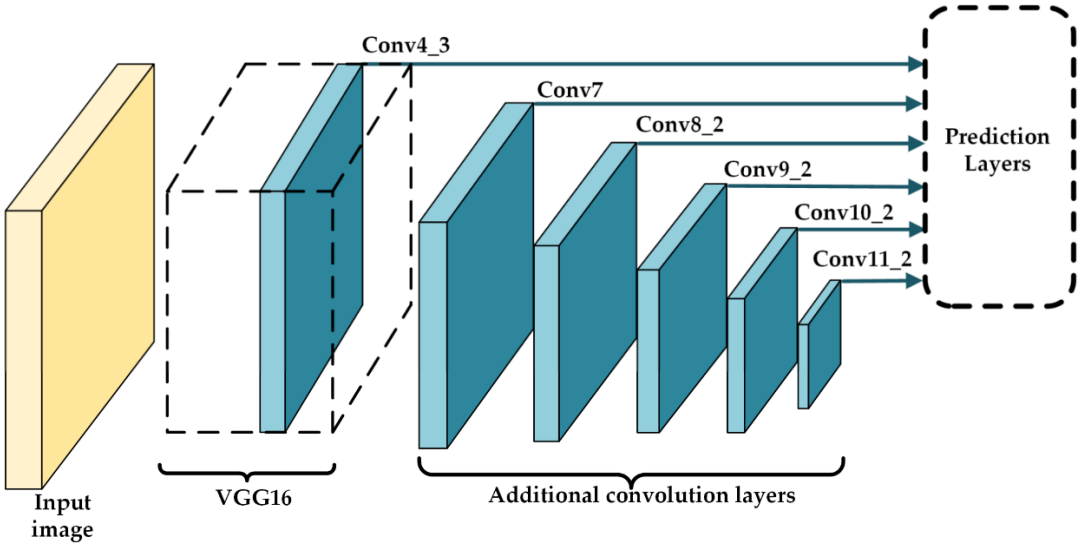

Figure 2.9 SSD architecture showing detection at multiple feature map scales (Li et al, 2021)

However, the trade-off in high accuracy is critical for this project. SSD struggles with small object detection because it relies on lower-resolution feature maps to predict objects by referring to the figure 2.9 (Liu et al., 2019). In the scenario of pickleball tracking, the ball appears as a very small object and prone to missed detections. Making it less reliable than the proposed YOLO solution.

2.5.3 You Only Look Once (Yolov11)

The You Only Look Once (YOLO) framework is a unified architecture designed to optimize object detection for real-time applications. Proposed by Redmon et al. (2016), YOLO redefines the traditional detection pipeline by framing the entire task as a single regression problem. The architecture operates by dividing the input image into a grid, as illustrated in Figure 2.10. Within this structure, each grid cell is responsible for simultaneously predicting bounding box coordinates, confidence scores, and class probabilities. This unified design allows the neural network to process the full image in a single forward pass, enabling high-speed inference without the need for complex, sequential processing steps (Redmon et al., 2016).

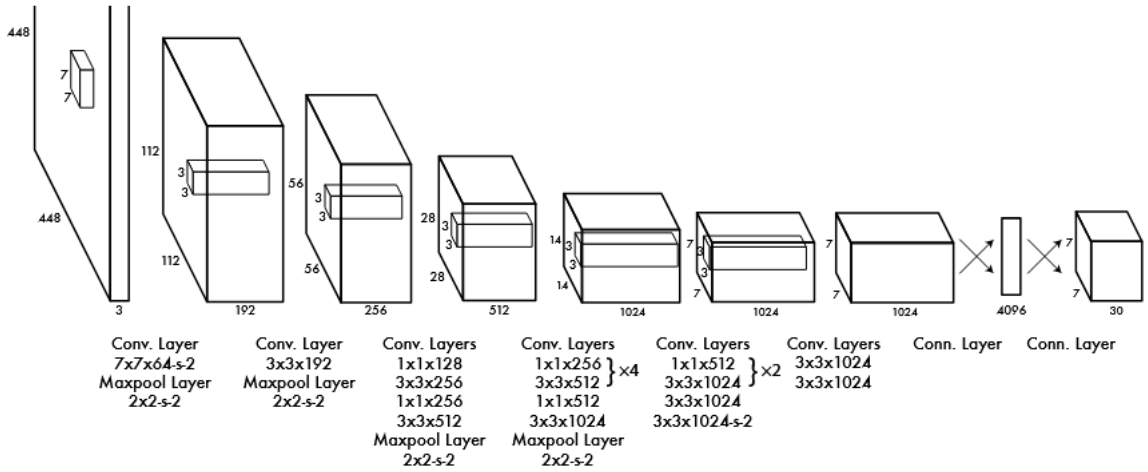

Figure 2.10 The YOLO network architecture (Redmon et al., 2016)

For this project, the YOLOv11-m (Medium) model was selected as the optimal architecture. Released by Ultralytics in late 2024, it introduces significant improvements suited for the dynamic environment of pickleball. The model utilizes enhanced feature extraction layers that allow it to capture invariant features, ensuring the ball is recognized even when its texture is degraded by motion blur or changing lighting conditions (Jocher et al., 2024).

Furthermore, YOLOv11 addresses the traditional weakness of one-stage detectors regarding small objects by employing an adaptive attention mechanism. This allows the network to dynamically scale its focus, which is vital for this project as the pickleball often occupies fewer than 20 x 20 pixels at the far end of the court. Finally, the medium variant provides a superior speed-accuracy trade-off compared to predecessors (v8 or v5), ensuring a consistent frame rate of more than 30 FPS on standard consumer hardware.

2.5.4 Comparison of Algorithms

To validate the selection of YOLOv11, Table 2.2 compares the three algorithms based on their suitability for real-time sports tracking on consumer hardware.

Table 2.2 Comparison of Object Detection Algorithms

Feature	**Faster R-CNN (Ren et al., 2017)**	**SSD (Liu et al., 2016)**	**YOLOv11 (Jocher & Qiu, 2024)**
Architecture	Two-Stage (Region Proposal + Detection)	One-Stage (Multi-scale Feature Maps)	One-Stage (Grid-based Regression)
Primary Strength	Highest Accuracy (mAP)	Good Speed/Accuracy Balance	Highest Speed (FPS)
Weakness	Slow inference speed; computationally heavy.	Struggles with very small objects.	Slightly lower accuracy on overlapping objects.
Inference Speed	Slow (~5–7 FPS)	Medium (~20–40 FPS)	Fast (~60+ FPS)
Suitability for Sports	Low (Too much latency for fast balls)	Medium (Good, but misses small balls)	High (Real-time tracking capable)

As shown in Table 2.2, Faster R-CNN is rejected due to its inability to run in real-time. SSD is rejected due to potential accuracy issues with small objects. YOLOv11 is selected as the optimal solution because it is the only architecture capable of achieving >60 FPS on a local PC, which is necessary to track the high-velocity trajectory of a pickleball without skipping frames.

2.6 Comparative Summary

This chapter provided a comprehensive review of the theoretical and technical foundations required to develop an Automated Pickleball Point Counting System. The investigation began by analyzing the complexities of the pickleball side-out scoring system, identifying the high cognitive load placed on recreational players.

A review of existing commercial solutions, including Hawk-Eye, SwingVision, and the In/Out device, revealed a significant gap in the market. While these systems offer advanced tracking, they remain inaccessible to the general community due to high infrastructure costs, subscription fees, or hardware exclusivity.

Technologically, the chapter evaluated various object detection architectures. While Two-Stage detectors like Faster R-CNN offer high accuracy, they lack the inference speed required for real-time tracking on consumer hardware. Consequently, YOLOv11 was identified as the optimal algorithm, offering the best balance of speed and accuracy for tracking high-velocity projectiles.

CHAPTER 3 : REQUIREMENT ANALYSIS AND DESIGN

3.1 Introduction

This chapter serves as the transition from the theoretical foundations and literature review established in Chapter 2 to the practical process of the project. While the previous chapter identified the gap in existing commercial solutions and justified the selection of the YOLOv11 algorithm, this chapter focuses on the detailed planning required to implement the solution.

The primary objective of this chapter is to define the requirement analysis and design specifications for the Automated Pickleball Point Counting System. This chapter is organized into three key sections. First, this chapter detailing the methodology. Second, this chapter presents the requirement analysis, specifying the functional and non-functional requirements. Finally, this chapter details the system design, illustrating the system architecture, database schema, and user interface components.

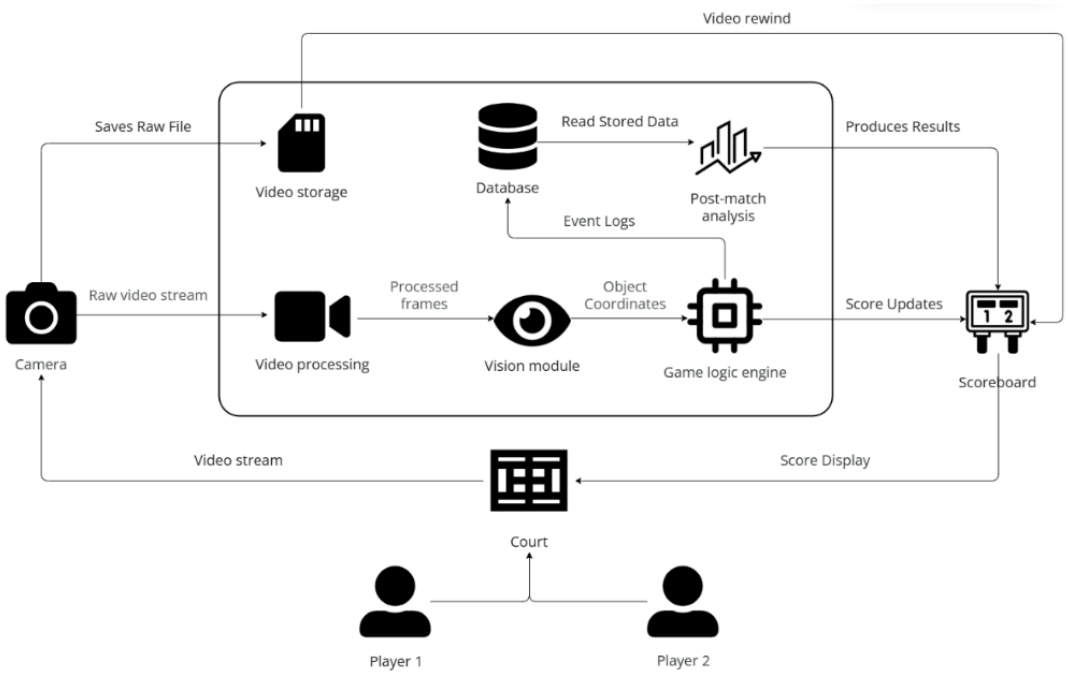

Figure 3.1 Proposed System Architecture

The high-level architecture of the proposed system is illustrated in Figure 3.1. This shows the beginning of the video input from the live camera feed, processed by the computer vision module to extract frame data. The detected coordinates are then passed to the game logic engine to determine the match state. Finally, the results are stored in the database and visualized on the dashboard interface for the user.

3.2 METHODOLOGY

The development of the Automated Pickleball Point Counting System adopts the Waterfall model. This approach organizes the project lifecycle into four different stages, which is the data collection and preparation, core detection module, game logic and database development and system integration and testing. This sequential approach ensures prioritization of the core Artificial Intelligence (AI) component before the development of the application logic. This ensures that the object detection model meets the necessary accuracy requirements before it is integrated into the final software solution. Figure 3.2 shows the waterfall model for this project.

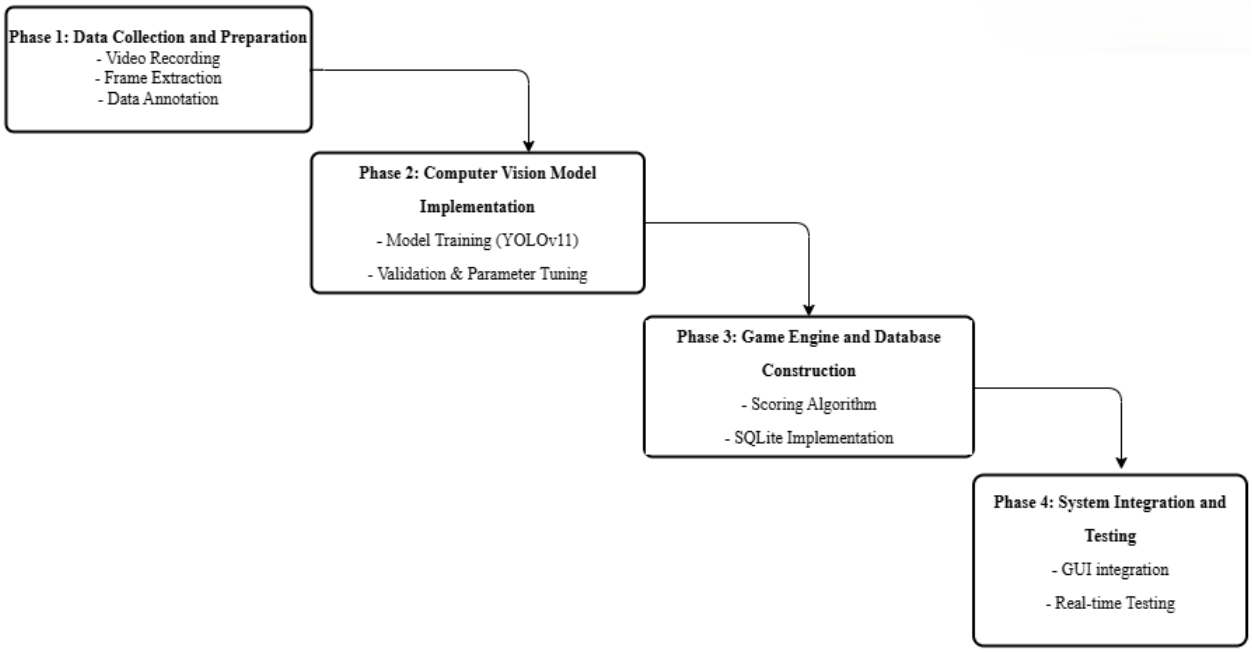

Figure 3.2 Development Framework of the Automated Pickleball Point Counting System

3.2.1 Phase 1: Data Collection and Preparation

The first phase focuses on a custom dataset. Data collection involves recording actual gameplay using a standard USB webcam set to a 1080p resolution. As illustrated in Figure 3.3, the camera is positioned at a fixed point at the rear center of the court, elevated approximately 1.5 meters high. This specific placement ensures a complete Field of View (FOV) of the court lines and player movements while minimizing occlusion.

This specific configuration was selected based on several key factors. The longitudinal angle offers an optimal Field of View (FOV) that captures both the serving and receiving sides within a single frame. Furthermore, positioning the camera at an elevated height minimizes the risk of obstruction by the players compared to net-level or side-line configurations. Finally, the fixed position of the camera creates a uniform tracking environment, ensuring the detection algorithm can consistently monitor the ball across the entire court.

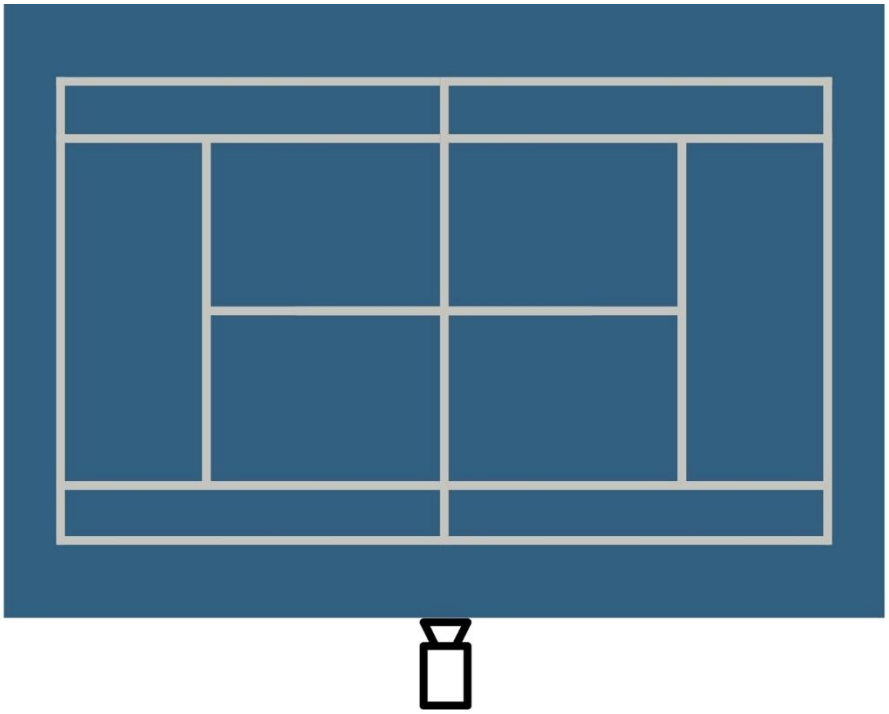

Figure 3.3 The position of camera at the rear baseline

Once the raw video footage is captured, it is processed into individual frames using the OpenCV library. These frames were manually annotated using tools of Roboflow, where bounding boxes were drawn around the pickleball in each frame to clearly define the object of interest. Preliminary validation of this annotation workflow was conducted using proxy objects, as shown in Figure 3.4, before proceeding to full-scale on-court data collection.

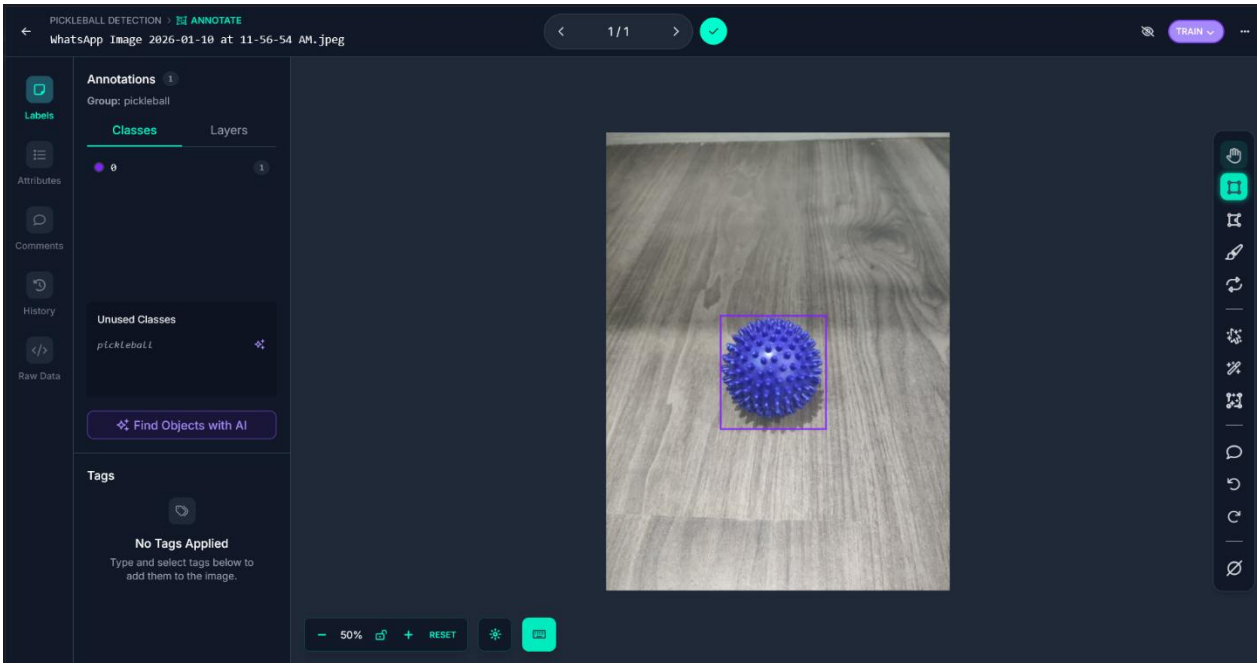

Figure 3.4 Illustration of the bounding box annotation workflow using a proxy object for preliminary testing

Finally, the annotated dataset was partitioned into three distinct subsets: Training Set of 70% for learning features, Validation Set of 20% for hyperparameter tuning during training, and Testing Set of 10% for the final unbiased evaluation.

3.2.2 Phase 2: Core Detection Module (Yolov11 Training)

In this phase, the training process focuses on maximizing the Mean Average Precision (mAP). The model is fine-tuned on the custom dataset prepared in Phase 1 using the PyTorch deep learning framework. mAP is a performance metric that calculates the average precision across various recall levels, providing a single score that balances false positives and false negatives. This metric is crucial for validating the model's consistency in detecting the small, high-velocity pickleball across different frames. Specifically, the system aims to optimize mAP@50, and hyperparameters will be iteratively adjusted to maximize this score, ensuring high-confidence tracking in real-time.

3.2.3 Phase 3: Game Logic And Database Development

A python language is used to construct a base game logic engine as a state machine. This engine is responsible for enforcing the specific rules of pickleball, particularly the traditional side-out scoring format, which requires tracking the serving teams score, the receiving teams score, and the server number. Concurrently, a local storage solution is implemented using the SQLite database engine. This database is designed to log game events accompanied by a precise timestamp. This ensures the system works entirely offline and preserves match data for post-game analysis.

3.2.4 Phase 4: System Integration And Testing

The final phase involves integrating the independent components into a unified functional prototype. The coordinates generated by the YOLOv11 module in phase 2 are fed directly into the

game logic engine developed in phase 3. The engine interprets these coordinates to update the game state. Consequently, a user dashboard is developed using standard web tools to display the live score and match status. The completed system then undergoes validation testing, where the automated point counting accuracy is compared against manual scorekeeping to evaluate its effectiveness and reliability in a real-world scenario.

3.3 Requirements Analysis

This section classifies the system specifications into two categories which are functional requirements and non-functional requirements. The functional requirements are the core operations of the system. Non-Functional Requirements specify the quality attributes. These requirements serve as the validation criteria for the final prototype. To visualize the structural hierarchy and grouping of these requirements, the system modeling is illustrated in Figure 3.5.

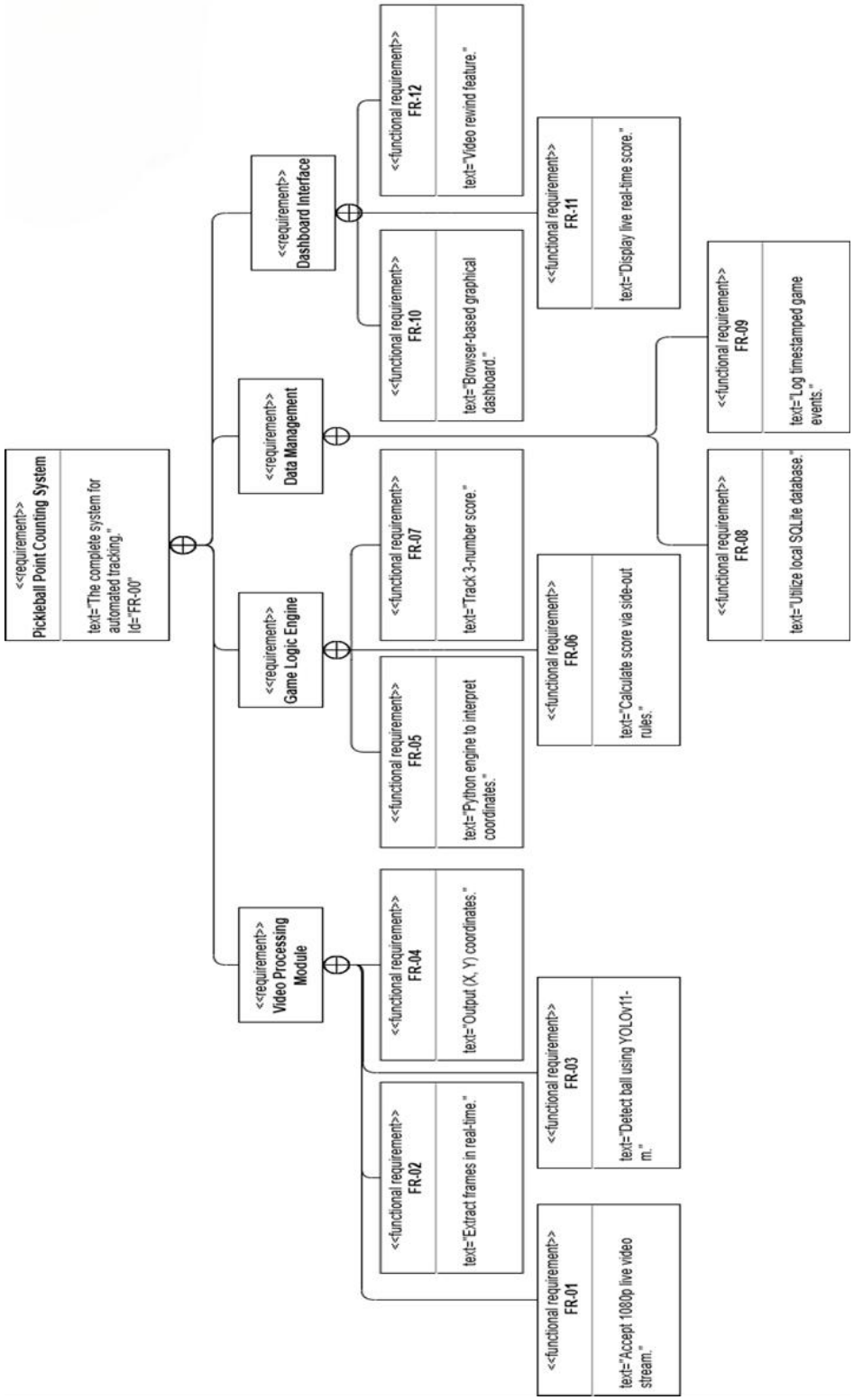

Figure 3.5 System Modeling Language Diagram

3.3.1 Functional Requirements

Functional requirements define the specific behaviors, operations, and functions that the system must execute to satisfy the project objectives. These requirements have been derived from the project scope and the official rules of pickleball side-out scoring. Table 3.1 shows the functional requirements list of the project.

Table 3.1 Table 3.1 List of Functional Requirements

ID	Use Case	Requirement Description	Priority
FR-01	Capture Video Stream	The system shall accept a continuous live video stream from a standard USB webcam.	High
FR-02	Capture Video Stream	The system shall process the raw video stream by extracting individual frames in real-time.	High
FR-03	Process Object Detection	The system shall utilize the YOLOv11-m deep learning model to detect the pickleball in video frames.	High
FR-04	Process Object Detection	The system shall output the geometric coordinates of the detected ball for every frame.	High
FR-05	Update Game State	The system shall implement a Python-based logic engine to interpret the coordinates of the ball.	High
FR-06	Update Game State	The system shall automatically calculate the score using traditional side-out scoring rules.	High
FR-07	Update Game State	The system shall track the three-number score	High

FR-08	Save Match Log	The system shall utilize a local SQLite database to store game data and logs.	Medium
FR-09	Save Match Log	The system shall generate a timestamped log for every significant game event.	Medium
FR-10	View Live Dashboard	The system shall provide a graphical dashboard interface viewable in a standard web browser.	Medium
FR-11	View Live Dashboard	The dashboard shall display the live match score to the users in real-time.	High
FR-12	Rewind Video	The dashboard shall include a video rewind feature to replay recent footage for verification.	Low

3.3.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes of the system, such as performance, reliability, and constraints. These ensure the system is usable and effective in a real-world scenario. Table 3.2 details the non-functional requirements for the project.

Table 3.2 List of Non-Functional Requirements

ID	Category	Requirement Description	Priority
NFR-01	Performance	The system must process video at a minimum of 60 FPS to track high-speed balls.	High
NFR-02	Constraint	The system must operate entirely on a local PC without requiring internet connectivity.	High
NFR-03	Accuracy	The object detection model shall be optimized to maximize mean Average Precision (mAP) for small objects.	High
NFR-04	Constraint	The system must utilize standard consumer hardware to ensure low cost.	High
NFR-05	Usability	The user interface shall be designed for simplicity, requiring minimal configuration from the user.	Medium
NFR-06	Reliability	The system must be robust against environmental noise such as variable lighting or motion blur.	Medium

3.3.3 Hardware and Software Requirements

To ensure the system meets the real-time performance and offline processing constraints defined in the non-functional requirements, the following hardware and software environments are specified in Table 3.3 and Table 3.4.

Table 3.3 Hardware Requirements

Component	Specification
Development PC	Laptop • OS: Windows 10/11 • RAM: 16GB • GPU: NVIDIA GTX/RTX Series
Camera	USB Webcam • Resolution: 1080p • Frame Rate: 60 FPS
Sport Equipments	Standard Pickleball Gear • Paddles • Polymer Balls

A high-performance laptop is selected as the primary workstation for coding and model training. A dedicated NVIDIA GPU is recommended since it provide CUDA acceleration for the PyTorch framework. This acceleration is critical to ensuring the YOLOv11 model can process video frames fast enough to exceed the 30 FPS threshold defined in NFR-01.

A standard USB webcam is chosen to provide the raw video input for the Vision Module, ensuring plug-and-play compatibility. However, a specific frame rate of 60 FPS is selected to minimize motion blur on high-velocity shots, which is necessary for accurate tracking.

Standard pickleball gear, including paddles, polymer balls, is required to record the custom dataset for training and to perform real-world validation of the point counting accuracy of system.

Table 3.4 Software Requirements

Component	**Tools**
Language	Python 3.9+
AI Framework	PyTorch
Vision Library	OpenCV
Database	SQLite
Data Annotation	Roboflow
Frontend	HTML, CSS, JavaScript

The core software architecture is built using Python 3.9+, selected as the primary development language to develop the backend game logic and system integration. For the computer vision component, OpenCV is used for capturing the live webcam stream and preprocessing raw video frames before detection. PyTorch framework is used to fine-tune and execute the YOLOv11-m object detection model with the leveraging GPU acceleration to ensure real-time performance.

Roboflow is use for labeling the custom dataset and generating the necessary training and validation splits. SQLite was selected as the database engine to store the necessary data without internet dependency. Finally, the user dashboard is rendered using standard web technologies.

3.4 System Design

The system design phase translates the functional and non-functional requirements into visual models that guide the implementation. This section details the systems design across three key layers, which are interaction design of use case and sequence diagrams, logic design of activity diagrams, and structural design of class diagrams, database schema, and GUI layouts. Collectively, these models provide a comprehensive blueprint for the development process.

3.4.1 Use Case Diagram

The diagram shows how the user and hardware components communicate with the system to process the live video stream. Figure 3.6 illustrates the use case diagram for the proposed Automated Pickleball Point Counting System.

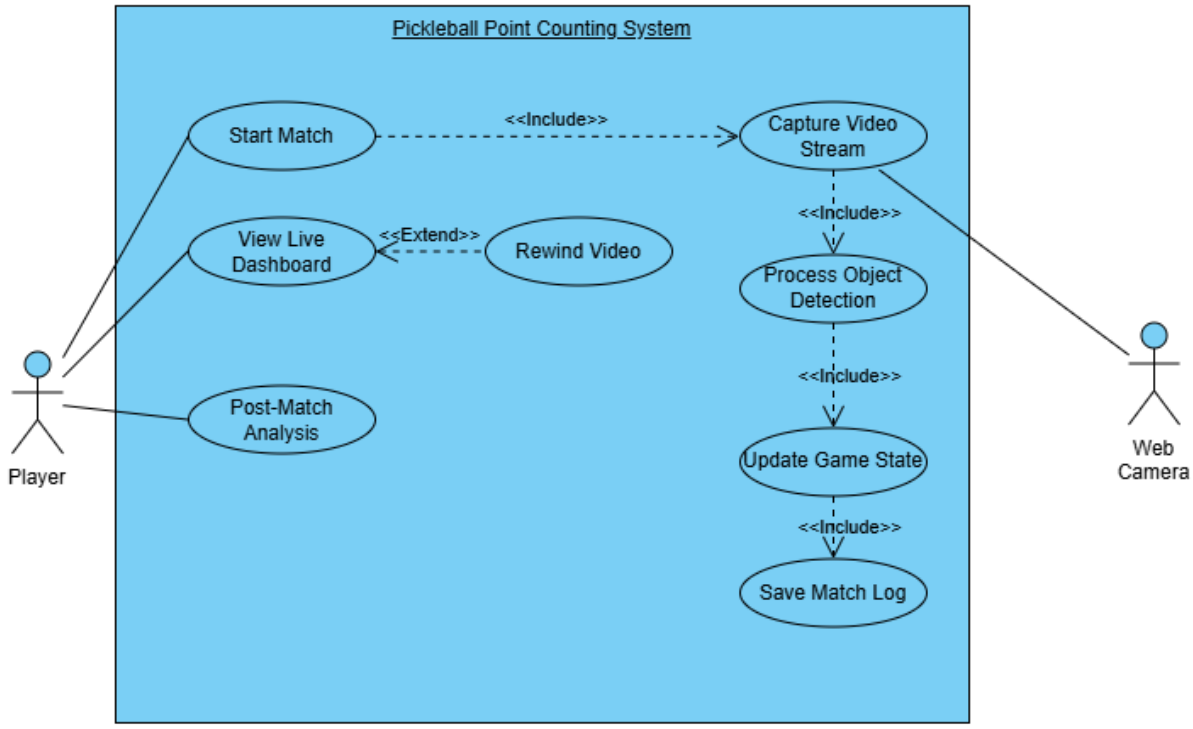

Figure 3.6 Automated Pickleball Point Counting System use case diagram

As shown in Figure 3.5, the system involved two primary actors, namely the player and the web camera. The player is involved in initiating the match, viewing live dashboard and post-match analysis. While the web camera serves as the external hardware actor that provides the video input for the object detection module.

3.4.2 Use Case Description

Table 3.5 Use Case Description for automated game tracking

Use Case Name	Capture Video Stream
Use Case ID	UC-01
Actor	Web Camera
Requirements	FR-01: System shall accept continuous live video stream. FR-02: System shall process raw video frames.
Description	The system continuously accepts raw video frames from the hardware device to initiate the tracking process.
Pre-conditions	1. User has initiated start match function. 2. Camera is connected via USB.
Post-conditions	A raw video frame is stored in memory and ready for processing.
Main Flow	1. The Web Camera activates and sends a video stream signal. 2. The system receives the signal and buffers the current frame. 3. The system passes the frame to the detection module.
Alternate Flow	A1: Camera Disconnected 1. The system detects the video signal is lost. 2. The system pauses the game logic. 3. The system displays a warning on the dashboard.
Exception Flow	E2: Initialization Failure 1. The system fails to open the camera driver. 2. The system logs a critical error and aborts the start match process.

Table 3.6 Use Case Description for Process Object Detection

Use Case Name	Process Object Detection
Use Case ID	UC-02
Actor	System
Requirements	FR-03: System shall utilize YOLOv11-m model. FR-04: System shall output geometric coordinates.
Description	The system analyzes the frame to locate the pickleball using the YOLOv11 deep learning model.
Pre-conditions	UC-01 has successfully buffered a raw frame.
Post-conditions	X and Y coordinates of the ball are identified.
Main Flow	1. The system resizes the frame to the models input size. 2. The YOLOv11 model scans the image for the ball class. 3. The system extracts the bounding box center coordinates.
Alternate Flow	A2: No Ball Detected 1. The model confidence score is below the threshold. 2. The system returns a null coordinate. 3. The flow ends for this specific frame.
Exception Flow	E1: Model Corruption 1. The system cannot load the yolo.pt weight file. 2. The system throws a model missing exception.

Table 3.7 Use Case Description for Update Game State

Use Case Name	Update Game State
Use Case ID	UC-03
Actor	System
Requirements	FR-05: System shall interpret ball coordinates. FR-06: System shall calculate score using side-out rules. FR-07: System shall track the three-number score.
Description	The logic engine applies pickleball rules to the position of the ball to update the score.
Pre-conditions	UC-02 has output valid ball coordinates.
Post-conditions	The score is updated in memory.
Main Flow	1. The engine maps the coordinates to the court zones. 2. The engine determines if the rally ended. 3. The system updates the three-number score accordingly.
Alternate Flow	A3: Match Concluded 1. The serving team reaches 11 points. 2. The system sets the game state to finish. 3. The system triggers the post-match prompt.
Exception Flow	E3: Logic Error 1. The game state becomes invalid. 2. The system resets the state to the last known valid score.

Table 3.8 Use Case Description for Save Match Log

Use Case Name	Save Match Log
Use Case ID	UC-04
Actor	System
Requirements	FR-08: System shall utilize local SQLite database. FR-09: System shall generate timestamped logs.
Description	The system records the game event to the database for persistence and future analysis.
Pre-conditions	UC-03 has completed a score update.
Post-conditions	A new row is inserted into the Game_Events table.
Main Flow	1. The system constructs a data object with Timestamp, Event Type, and Score. 2. The system executes an INSERT query to the SQLite database.
Alternate Flow	A2: Rapid Events 1. Multiple events occur within milliseconds. 2. The system queues them to ensure sequential writing.
Exception Flow	E2: Database Locked 1. The SQLite file is currently being read by the dashboard. 2. The system retries the write operation after 10ms.

Table 3.9 Use Case Description for View Live Dashboard

Use Case Name	View Live Dashboard
Use Case ID	UC-05
Actor	Player
Requirements	FR-10: System shall provide a graphical dashboard. FR-11: Dashboard shall display live match score.
Description	The user monitors the real-time match status, score, and server indicator.
Pre-conditions	The match is in progress and the system is running.
Post-conditions	The user is visually informed of the game state.
Main Flow	1. The user views the dashboard screen. 2. The system fetches the current score from memory. 3. The UI updates the scoreboard numbers and server indicator.
Alternate Flow	A3: Manual Override 1. The user notices a wrong automated call. 2. The user clicks undo on the dashboard. 3. The system reverts the score to the previous state.
Exception Flow	E3: UI Freeze 1. The browser/interface becomes unresponsive. 2. The user refreshes the page to reload the current state from the database.

Table 3.10 Use Case Description for Rewind Video

Use Case Name	Rewind Video
Use Case ID	UC-06
Actor	Player
Requirements	FR-12: Dashboard shall include Video Rewind feature.
Description	The user replays recent footage to verify a disputed call.
Pre-conditions	User is viewing the dashboard (UC-05).
Post-conditions	The video clip is replayed on the screen.
Main Flow	1. The user clicks the rewind button. 2. The system retrieves the last 10 seconds of video from the buffer. 3. The dashboard plays the clip in a modal window.
Alternate Flow	A3: Cancel Rewind 1. The user closes the replay window before it finishes. 2. The system immediately resumes the live feed.
Exception Flow	E2: Buffer Empty 1. The user attempts to rewind immediately after the match starts. 2. The system displays not enough footage notification.

3.4.3 Sequence Diagram

To visualize the real-time data exchange within the application, the sequence diagram Figure 3.7 details the lifecycle of a video frame from capture to point counting.

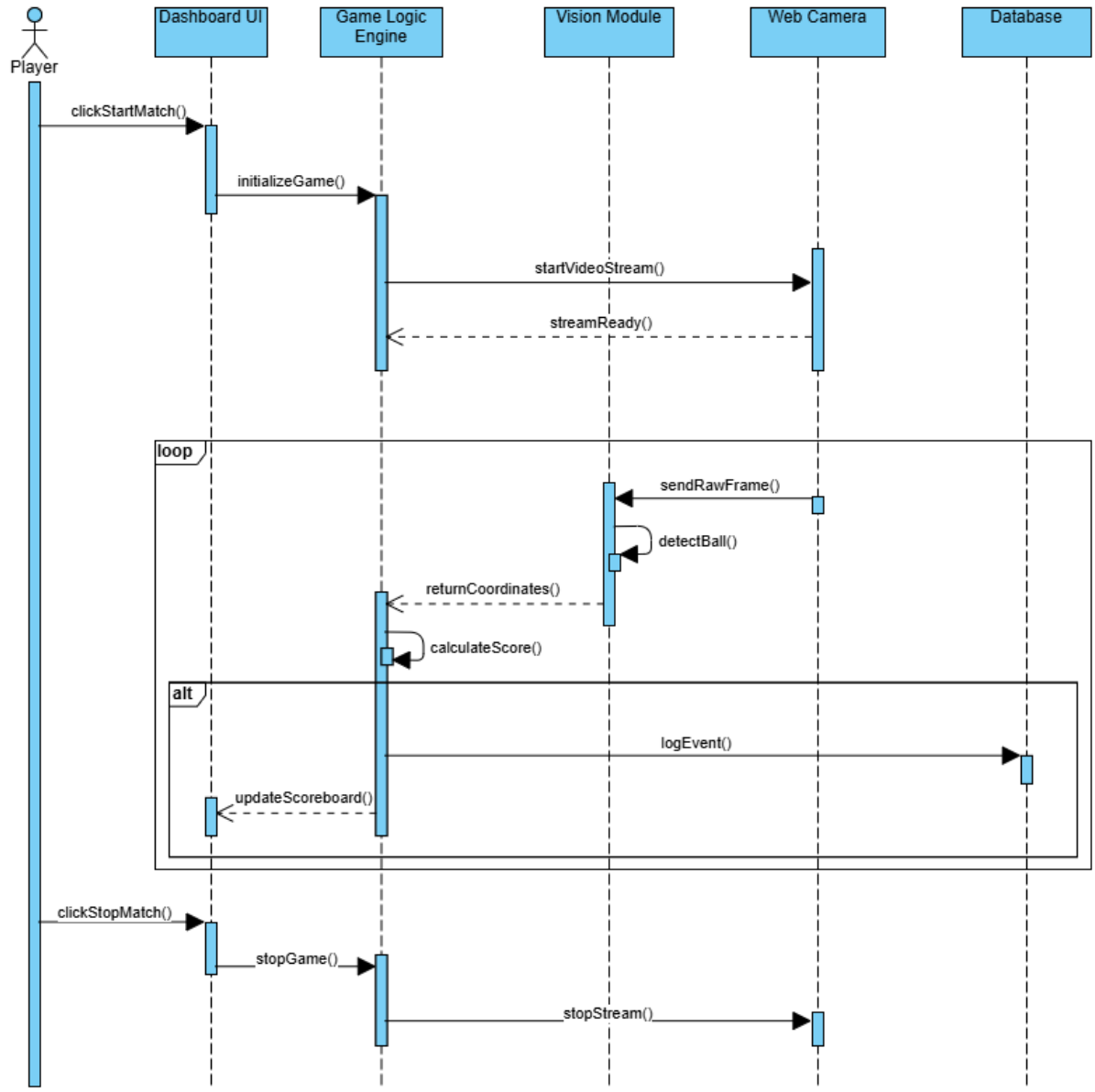

Figure 3.7 Sequence diagram of Automated Pickleball Point Counting System

The process is divided into three distinct phases, Initialization, the Main Processing Loop, and Termination. The workflow begins when the Player initiates the clickStartMatch() event on the

DashboardUI. This triggers the Game Logic Engine to initialize the game state and signal the Web Camera to begin capturing footage startVideoStream().

Once the stream is ready, the system enters a continuous loop cycle. Within this cycle, the camera continuously sends raw frames to the Vision Module, which executes the detectBall() algorithm. Upon detection, the coordinates are returned to the Game Logic Engine for interpretation through calculateScore().

The Alternative fragment shows a conditional logic, if a scoring event is determined, the engine simultaneously triggers logEvent() and updateScoreboard(). This loop continues until the player triggers the clickStopMatch() event, which safely terminates the video stream and saves the final session state.

3.4.4 Activity Diagram

The activity diagram shows the system of game logic engine by figure 3.8. It illustrates the sequential flow of operations required to enforce traditional side-out scoring rules (FR-06).

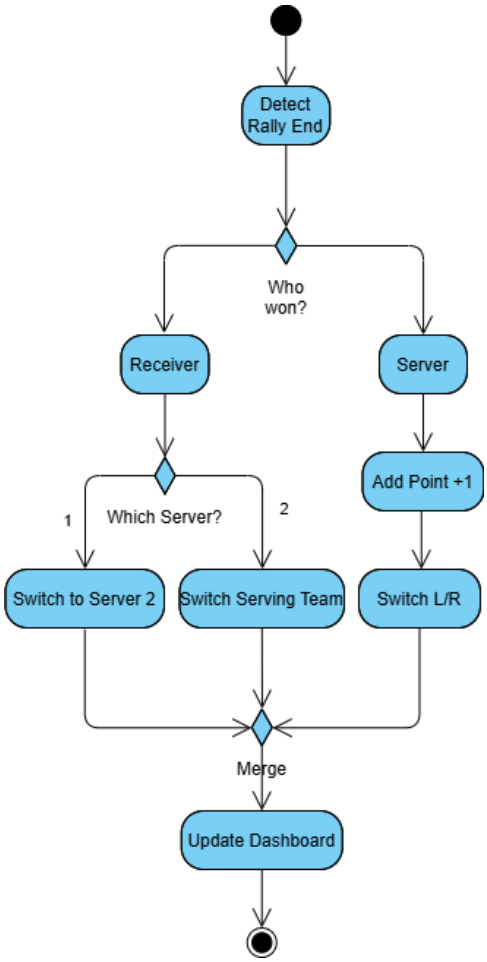

Figure 3.8 Activity Diagram of Pickleball Scoring Rules

The process is triggered automatically when the vision module identifies the termination of a rally. The logic flows make decision based on the teams, if the Serving Team wins the rally, it increments their score and updates the virtual player positions.

If the Receiving Team wins the rally, the system enters a nested decision branch to check the current Server Number (FR-07). If the current server is Player #1, the system executes Switch to Server 2, transferring the serve score. If the current server is Player #2, the system executes Switch Serving Team, resetting the server number to #1 and transferring service control to the

opponents. All logical paths merge to execute the update dashboard action, ensuring the user interface immediately reflects the calculated game state.

3.4.5 Class Diagram

The structural design and component interactions are illustrated in the figure 3.9.

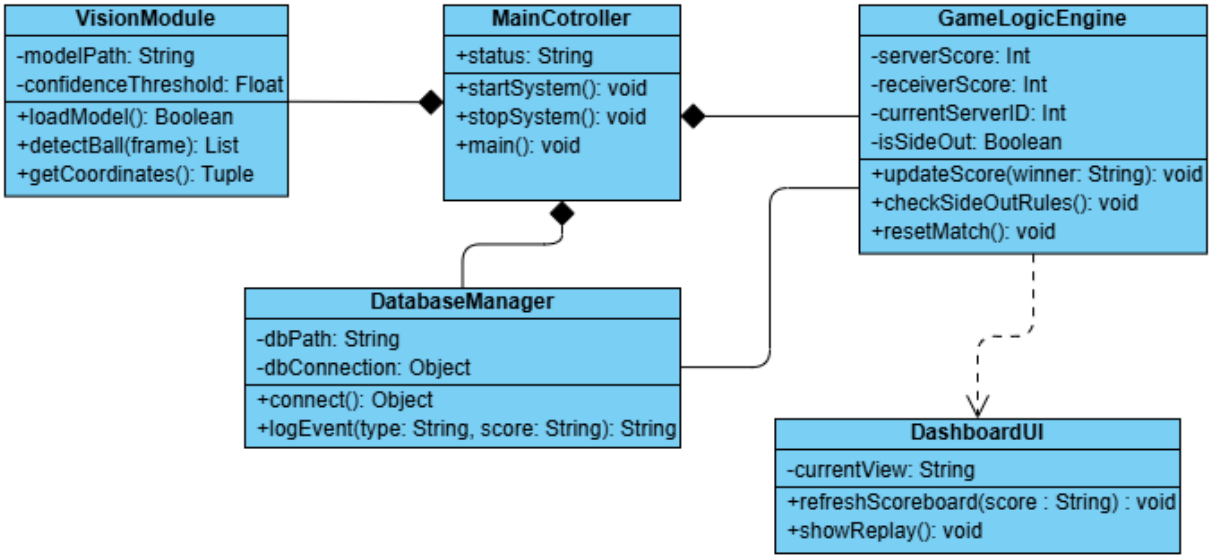

Figure 3.9 Class Diagram of the Automated Pickleball Point Counting System

The MainController act as an orchestrated for the application. This controller managing the subsystems, indicates by the composition relationships pointing to the main. If the main controller is terminated, these dependent subsystems are also properly closed.

VisionModule encapsulates the computer vision tasks, outputting coordinates to the controller. The GameLogicEngine maintains match states and enforcing side-out rules. It also maintains the direct association between the DatabaseManager, enabling the logic engine to trigger data logging events whenever a score changes. Finally, the DashboardUI is linked to the logic

engine through a dependency relationship. The dashboard interface only retrieves and displays the current score from the engine.

3.4.6 Entity Relationship Diagram (ERD)

The entity relationship figure 3.10 illustrates the database schema to support the offline data persistence (FR-08) and post-match analysis.

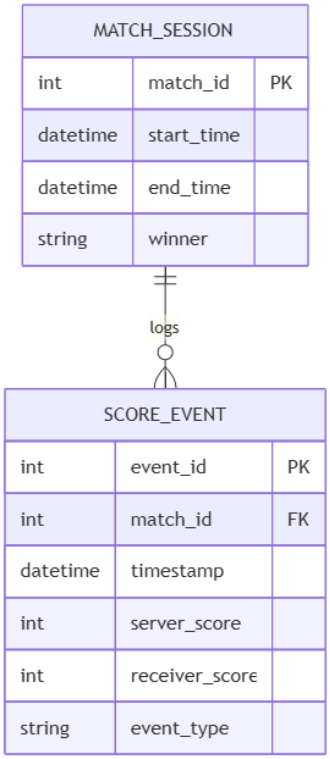

Figure 3.10 Entity Relationship Diagram of the System Database

The database design is organized into two main tables of MATCH_SESSION and SCORE_EVENT. The MATCH_SESSION entity serves as the parent record, storing match results for each game. It records the duration of the session and the winner. The SCORE_EVENT entity detailed a transaction log that occurs during gameplay (FR-09). It captures the timestamp, the specific event_type, and the state of the score at that moment. The relationship between these two entities is a one-to-many association, indicated by the crow foot notation. A single match session generates multiple score events, which are linked back to the parent session through the match_id foreign key.

3.4.7 Graphical User Interface (GUI) Design

Figure 3.11 illustrates the Graphical User Interface (GUI) design for the Automated Pickleball Point Counting System.

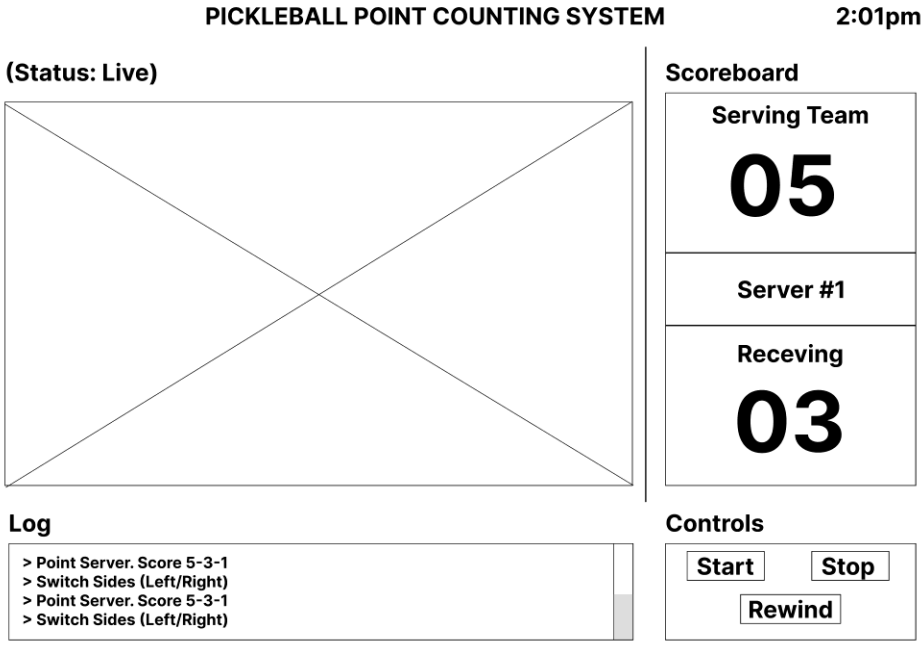

Figure 3.11 Graphical User Interface of scoreboard

The central section of the screen is reserved for the live video feed, which displays the real-time stream processed by the vision module. The right side is occupied by the dynamic scoreboard, displaying the current scores for both the serving and receiving teams. Lastly, the section below the screen features a logging panel to display timestamped game events, alongside a control panel that allows users to replay the last few seconds of gameplay to verify disputed calls.

3.5 Summary

In summary, this chapter covers the methodology, requirements analysis and system design for the proposed system. The development process adopts a Waterfall model approach, which prioritizes the training and validation of the YOLOv11-m computer vision module. The requirements analysis defined the functional and non-functional specifications of the system. Furthermore, the system design visualized the overall design architecture through various system modeling diagrams. The implementation part will be continued in the next chapter, where these specifications will be realized through code development and model training.

REFERENCES

Faiza, B. (2025). Hawk-Eye explained: The $100k Tech Powering Global Sports. Trustnbet.com. https://www.trustnbet.com/sports-news/hawk-eye-explained-the-100k-tech-thats-quietly-running-global-sports/

Ghoury, S., Sungur, C., & Durdu, A. (2019). Real-Time Diseases Detection of Grape and Grape Leaves using Faster R-CNN and SSD MobileNet Architectures. *International Conference on Advanced Technologies, Computer Engineering and Science (ICATCES 2019)*. https://www.researchgate.net/publication/334987612_Real-Time_Diseases_Detection_of_Grape_and_Grape_Leaves_using_Faster_R-CNN_and_SSD_MobileNet_Architectures

Hawk-Eye Innovations. (n.d.). Hawk-Eye Electronic Line Calling System [Computer software]. https://www.hawkeyeinnovations.com/

In/Out Tennis. (n.d.). In/Out Net Device (Version 4.0) [Apparatus]. https://inout.tennis/

Jiao, L., Zhang, F., Liu, F., Yang, S., Li, L., Feng, Z., & Qu, R. (2019). A Survey of Deep Learning-Based Object Detection. *IEEE Access*, *7*, 128837–128868. https://doi.org/10.1109/access.2019.2939201

Jocher, G., & Qiu, J. (2024). Ultralytics YOLO11 (Version 11.0.0) [Computer software]. Ultralytics. https://github.com/ultralytics/ultralytics

Li, Q., Lin, Y., & He, W. (2021). SSD7-FFAM: A Real-Time Object Detection Network Friendly to Embedded Devices from Scratch. *Applied Sciences*, *11*(3), 1096. https://doi.org/10.3390/app11031096

Liu, L., Ouyang, W., Wang, X., Fieguth, P., Chen, J., Liu, X., & Pietikäinen, M. (2019). Deep Learning for Generic Object Detection: A Survey. *International Journal of Computer Vision*, *128*(2), 261–318. https://doi.org/10.1007/s11263-019-01247-4

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., & Berg, A. C. (2016). SSD: Single Shot MultiBox Detector. In B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds.), Computer Vision – ECCV 2016 (pp. 21–37). Springer International Publishing. https://doi.org/10.1007/978-3-319-46448-0_2

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788. https://doi.org/10.1109/cvpr.2016.91

Ren, S., He, K., Girshick, R., & Sun, J. (2017). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *39*(6), 1137–1149. https://doi.org/10.1109/tpami.2016.2577031

SwingVision. (2025). SwingVision: Tennis Pickleball (Version 11.9.48) [Mobile app]. App Store. https://apps.apple.com/us/app/swingvision-tennis-pickleball/id989461317

Zhao, Z.-Q., Zheng, P., Xu, S.-T., & Wu, X. (2019). Object Detection With Deep Learning: A Review. *IEEE Transactions on Neural Networks and Learning Systems*, *30*(11), 3212–3232. https://doi.org/10.1109/tnnls.2018.2876865

Ziegler, M. (2013, August 8). *Hawk-Eye inventor Paul Hawkins says cricket didn't test system*. Daily Mail. https://www.dailymail.co.uk/sport/cricket/article-2386856/Hawk-Eye-inventor-Paul-Hawkins-says-cricket-didnt-test-system.html